



Dokumentace projektu z předmětu ISA
Monitorování DHCP komunikace

Brno, 20. listopadu 2023

David Kvaček
xkvace00@stud.fit.vutbr.cz

Obsah

1	Uvedení do problematiky	2
1.1	Obecný popis aplikace	2
1.2	Protokol DHCP	2
2	Návrh aplikace	4
2.1	Architektura aplikace	4
2.2	Princip fungování	4
3	Popis implementace	5
3.1	Implementační detaily	5
3.2	Struktura programu	5
3.2.1	Zdrojový soubor <code>dhcp-stats.c</code>	5
3.2.2	Hlavičkový soubor <code>dhcp-stats.h</code>	6
3.2.3	Funkční soubor <code>Makefile</code>	7
3.3	Zajímavé pasáže implementace	7
3.3.1	Funkce <code>within_range()</code>	7
3.3.2	Funkce <code>packet_handler()</code>	7
3.4	Funkční omezení	8
4	Návod na použití	9
4.1	Použití	9
4.2	Status ukončení	9
4.3	Příklad spuštění	9
4.4	Výstup	9
4.5	Logování	9
5	Seznam odevzdaných souborů	10
6	Literatura	11

1 Uvedení do problematiky

Tato technická zpráva poskytuje podrobný pohled na návrh a implementaci monitorovací aplikace DHCP komunikace. Je zaměřena na analýzu síťové komunikace, konkrétně na protokol DHCP [2], a přináší návrh efektivního nástroje pro sledování využívání adresového prostoru. Dokumentace zahrnuje popis architektury aplikace (2.1), postup implementace (3) a očekávané výstupy (4.4). Cílem je poskytnout užitečný nástroj pro správce sítí s důrazem na optimalizaci a bezpečnost síťové infrastruktury.

1.1 Obecný popis aplikace

Program **dhcp-stats** je uživatelská utilita umožňující získat statistiku o vytížení síťového prefixu z pohledu množství alokovaných IP adres. Při zaplnění prefixu z více jak 50%, 75%, 90% a 95%, nástroj informuje administrátora na standardní výstup a zalogováním skrz **syslog** server [4].

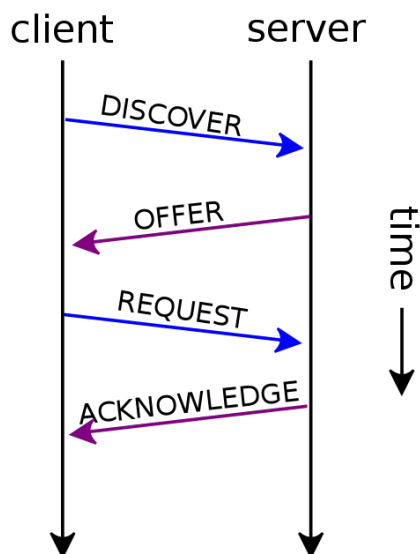
Program po spuštění začne monitorovat DHCP provoz na zvoleném rozhraní, případně zpracuje *pcap* soubor, a generovat si statistiku vytížení síťového prefixu, který mu byl zadán v příkazové řádce. Prefixů může být více a mohou se překrývat – důvodem možného překryvu je zjištění, jak by vypadalo vytížení sítě, kdyby byl prefix větší.

1.2 Protokol DHCP

Dynamic Host Configuration Protocol (DHCP) je síťový protokol používaný k automatické konfiguraci síťových zařízení. Jeho hlavním účelem je přidělovat IP adresy a další síťové konfigurační informace zařízením v síti.

DHCP funguje na klient-server modelu, kde DHCP server je zodpovědný za přidělování konfiguračních informací klientům v síti. Klienti, kteří připojí nové zařízení do sítě, komunikují s DHCP serverem za účelem získání IP adresy, výchozí brány, DNS serveru a dalších informací potřebných pro správnou komunikaci v síti.

Proces začíná tím, že klient vysílá **DHCPDISCOVER** zprávu na síť, aby informoval o své přítomnosti. DHCP server odpovídá zprávou **DHCPOFFER**, ve které nabízí konfigurační informace, včetně přidělované IP adresy. Klient poté posílá **DHCPREQUEST**, potvrzující přijetí nabízených informací. Nakonec server odpovídá **DHCPACK**, potvrzující přidělení konfigurace klientovi [13].



Obrázek 1: Ilustrace typické DHCP komunikace [13]

Protokol DHCP je klíčový pro správu síťových adres v dynamických sítích, umožňuje snadnou a efektivní konfiguraci nových zařízení a minimalizuje potřebu manuálního zásahu do síťových nastavení. Tato automatizace zvyšuje efektivitu správy sítě a eliminuje riziko konfliktů IP adres, což je zásadní pro plynulý provoz moderních síťových infrastruktur.

Služby DHCP existují pro sítě s internetovým protokolem verze 4 (IPv4) i verze 6 (IPv6). Verze protokolu DHCP pro protokol IPv6 se běžně nazývá DHCPv6 [13].

3 Popis implementace

Implementace monitorování DHCP komunikace pro sledování vytíženosti zadaných IP prefixů vyžaduje integrovaný přístup k síťové infrastruktuře a systému, který dovoluje zachytávání a analýzu DHCP paketů. Celý program je navržen tak, aby umožnil sledování DHCP provozu na síti a detekci vytížení definovaných IP prefixů. Program je rozdělen do několika klíčových částí, které zajišťují efektivní a bezproblémový chod.

3.1 Implementační detaily

Utilita **dhcp-stats** je napsána v programovacím jazyce C. Pro práci se soubory *pcap* a monitorování síťových rozhraní je v implementaci použita knihovna *libpcap* [7]. Pro *syslog* se používá standardní logovací rutina *syslog* [4]. Program **dhcp-stats** také předpokládá, že soubor *pcap* nebo síťové rozhraní bude mít k dispozici kompletní DHCP komunikaci, tj. jako kdyby byl nástroj spuštěn přímo na DHCP serveru.

3.2 Struktura programu

Veškerý zdrojový kód potřebný pro snadné spuštění utility se nachází ve třech souborech:

- **dhcp-stats.c** – zdrojový soubor v programovacím jazyce C.
- **dhcp-stats.h** – hlavičkový soubor v programovacím jazyce C.
- **Makefile** – funkční soubor pro překlad zdrojového souboru.

3.2.1 Zdrojový soubor dhcp-stats.c

Zdrojový soubor **dhcp-stats.c** obsahuje vlastní implementaci programu. Pro přehlednost je kód uspořádán do několika jednoduchých funkcí, které zajišťují funkčnost a chod celého nástroje. Hlavní funkce programu `int main(int argc, char *argv[])` zpočátku provede kontrolu argumentů zadaných prostřednictvím příkazové řádky pomocí funkce `getopt()` [6].

Pokud tato funkce skončí úspěšně, přichází na řadu validace zadaných IP prefixů. Tuto kontrolu má na svědomí jedna z dalších funkcí `bool cidr_validate(char *ip_prefix)`. Jak již z prototypu funkce vyplývá, její návratová hodnota je `true`, nebo `false` a na základě tohoto výsledku je zřejmé, zda-li zadaný IP prefix splňuje podmínku *CIDR* notace [12].

V tomto okamžiku je vše připraveno pro samotný běh programu. Nejprve se alokuje odpovídající počet struktur sloužících k uchovávání statistik jednotlivých prefixů podle jejich množství. Inicializuje se standardní logovací rutina *syslog* [4] a okno *ncurses* [8] pro práci s terminálem.

Následuje podstatná část programu, ve kterém se tok větví na dvě části. Pokud byl zadán jako parametr programu *pcap* soubor pomocí přepínače `-r`, nástroj zavolá odpovídající funkci, která zpracuje zadaný soubor – `pcap_t *pcap_open_offline(const char *, char *)`. Druhou možností je funkce `pcap_t *pcap_open_live(const char *, int, int, int, char *)`, která slouží k zachytávání paketů v reálném čase v případě zadaného rozhraní přepínačem `-i`.

Pro analýzu a detekci jednotlivých příchozích paketů v rámci síťového provozu je použita tzv. *callback* funkce `int pcap_loop(pcap_t *, int, pcap_handler, u_char *)`. Jakmile je *pcap* soubor zpracován nebo je ukončeno naslouchání na určitém rozhraní, program adekvátně ukončí všechny běžící procesy, zavře terminálové okno *ncurses* [8], vypíše celkové statistiky na standardní výstup, uvolní veškerou alokovanou paměť, uzavře standardní logovací rutinu *syslog* [4] a skončí očekávaným návratovým kódem, případně, pokud nastala chyba, vypíše chybovou hlášku na standardní chybový výstup.

3.2.2 Hlavičkový soubor dhcp-stats.h

Hlavičkový soubor `dhcp-stats.h` obsahuje deklarace funkcí a struktur pro implementaci uživatelské utility **dhcp-stats**, stejně jako všechny použité knihovny a makra. Soubor poskytuje rozhraní pro integraci s hlavním programem, udržuje přehled o využívaných funkcích a použitých strukturách a hraje klíčovou roli při oddělení rozhraní a implementace, což usnadňuje čitelnost a udržitelnost kódu.

Vzhledem k nejednoznačnosti ve standardním hlavičkovém souboru `<netinet/udp.h>` [3] je použita vlastní struktura pro UDP hlavičku. Tato struktura obsahuje standardní informace sloužící k identifikaci zdrojového a cílového portu, délky hlavičky a kontrolní součet. Deklarace struktury vypadá následovně:

```
/**
 * @brief Structure for UDP header.
 *
 */
struct udp_hdr {
    u_int16_t uh_sport;
    u_int16_t uh_dport;
    u_int16_t uh_ulen;
    u_int16_t uh_sum;
};
```

Aby bylo možné dále pracovat přímo s jednotlivými položkami daného DHCP paketu, je zapotřebí deklarovat strukturu DHCP hlavičky následovně:

```
/**
 * @brief Structure for DHCP header.
 *
 */
struct dhcp_hdr {
    u_int8_t op;
    u_int8_t htype;
    u_int8_t hlen;
    u_int8_t hops;
    u_int32_t xid;
    u_int16_t secs;
    u_int16_t flags;
    struct in_addr ciaddr;
    struct in_addr yiaddr;
    struct in_addr siaddr;
    struct in_addr giaddr;
    u_int8_t chaddr[CHADDR];
    char sname[SNAME];
    char file[FILE];
    u_int32_t magic_cookie;
    u_int8_t options[OPTIONS];
};
```

Struktura pro uchovávání statistik jednotlivých prefixů je neméně důležitá. Obsahuje nejen samotný IP prefix v podobě textového řetězce pro jednoduché tisknutí statistik na standardní výstup, ale také maximální počet adres, které lze pro daný prefix alokovat, aktuální počet alokovaných adres a samozřejmě procento, které udává aktuální vytížení IP prefixu v podobě desetinného čísla. Deklarace této struktury vypadá následujícím způsobem:

```

/**
 * @brief Structure for DHCP statistics.
 *
 */
struct dhcp_stats {
    char *ip_prefix;
    u_int32_t max_hosts;
    u_int32_t allocated_addresses;
    double utilization;
};

```

3.2.3 Funkční soubor Makefile

Makefile je konfigurační soubor pro sestavování programu. Cíle v tomto souboru popisují, jak překladač a linker mají sestavit projekt. Základní cíle obvykle zahrnují kompilaci zdrojových souborů, vytvoření spustitelného souboru a čištění dočasných souborů. Proměnné jsou použity pro definici překladače, volby překladače a dalších parametrů.

Makefile také obsahuje pravidla pro závislosti mezi soubory, což zajišťuje efektivní sestavení pouze změněných částí. Jednoduchý, ale mocný nástroj, který zjednodušuje proces sestavování a udržuje projekt dobře organizovaný.

Tento specifický **Makefile** slouží k sestavení programu **dhcp-stats** pomocí veřejně známého příkazu **make**. Kompilace se provádí pomocí překladače **gcc** [5] s nastavenými standardy **std=gnu17**, přičemž všechna varování jsou striktně brána jako chyby (**-Wall -Werror -Wextra -pedantic**). Program využívá knihoven **-lpcap**, **-lm** a **-lncurses**. Cílovým souborem je **dhcp-stats**, který je generován z **dhcp-stats.c**.

Makefile obsahuje také cíle pro vytvoření archivu zdrojových souborů a dokumentace ve formátu **tar** **make tar**. Pro čištění souborů kompilace a archivu ve formátu **tar** je k dispozici cíl **make clean**.

3.3 Zajímavé pasáže implementace

Projekt je členěn do několika souborů a jednotlivé soubory do několika jednoduchých a přehledných funkcí. I přesto lze představit dvě zajímavé funkce celého programu.

3.3.1 Funkce `within_range()`

Funkce `bool within_range(struct in_addr, char *)` slouží k ověření, zda zadaná IP adresa patří do daného rozsahu definovaného IP prefixem. Funkce přijímá strukturu `struct in_addr` reprezentující adresu a řetězec `char *`, obsahující IP adresu a masku ve formátu *adresa/maska*.

Funkce nejprve parsováním získává IP adresu a masku z prefixu. Poté převádí IP adresu na síťovou adresu. Následně vypočítá adresu sítě a broadcastovou adresu pomocí masky. Dalším krokem je porovnání zadané adresy s vypočítaným rozsahem a vrácení `true`, pokud adresa patří do rozsahu, v opačném případě `false`.

V případě, že maska je minimální (/0), funkce kontroluje, zda adresa není adresa sítě nebo broadcastová adresa. Pro masku maximální (/32) vrací vždy `false`. Pro ostatní masky porovnává, zda zadaná adresa leží mezi vypočítanou adresou sítě a broadcastovou adresou.

3.3.2 Funkce `packet_handler()`

Funkce `void packet_handler(u_char *, const struct pcap_pkthdr *, const u_char *)` slouží k analýze paketů zachycených pomocí knihovny **libpcap** [11]. Nejprve zjišťuje, zda se jedná o paket s protokolem IP a UDP [3], přičemž následně ověřuje, zda je port DHCP serveru 67 a cílový port DHCP klienta 68 a také zda paket obsahuje validní položku `magic_cookie` [2].

Pokud jsou splněny všechny podmínky, funkce kontroluje, zda není nově zjištěná adresa již přidělená. Pokud ne, přidá ji do seznamu přidělených adres a aktualizuje statistiky pro odpovídající IP

prefix. V případě překročení určitých procentuálních limitů (50%, 75%, 90% a 95%) využití přidělených adres v rámci daného prefixu, funkce generuje systémový log pomocí standardní logovací rutiny `syslog` [4] a vypisuje upozornění na standardní výstup.

Nakonec funkce aktualizuje a zobrazuje tabulku s aktuálními statistikami využití IP prefixů, což poskytuje přehled o maximálním počtu alokovatelných adres, počtu aktuálně přidělených adres a procentuálním využití jednotlivých prefixů.

Tato funkce je volána jako tzv. *callback* při každém zachyceném paketu během monitorování síťového provozu a slouží ke sledování a zaznamenávání statistik ohledně přidělovaných IP adresách DHCP serverem a jedná se v podstatě o hlavní jádro celého programu.

3.4 Funkční omezení

- Nástroj **dhcp-stats** nepodporuje adresy IPv6 a komunikaci DHCPv6.
- Zprávy DHCPRELEASE, DHCPDECLINE a aktualizace obsazenosti síťových prefixů na základě vypršení *lease-time* nejsou podporovány.
- Utilita **dhcp-stats** také předpokládá DHCP komunikaci na standardních portech 67 a 68.

4 Návod na použití

Minimální konfigurace pro úspěšný překlad a spuštění programu nezbytně vyžaduje následující prvky:

- GNU kompilátor (`gcc`) ve verzi podporující standard `gnu17` [5].
- Balíček `libpcap` pro zachytávání paketů (`<pcap.h>`) [7].
- Knihovna `-lm` pro matematické operace (`<math.h>`) [10].
- Knihovna `-lncurses` pro práci s terminálem (`<ncurses.h>`) [8].

4.1 Použití

```
$ ./dhcp-stats [-h] [-r <filename>] [-i <interface-name>] <ip-prefix> [ ... ]
```

`-h` – zobrazení nápovědy k použití na standardní výstup a ukončení.

`-r <filename>` – statistika bude vytvořena z *pcap* souboru.

`-i <interface-name>` – rozhraní, na kterém může program naslouchat.

`<ip-prefix>` – rozsah sítě, pro které se bude generovat statistika.

4.2 Status ukončení

Program **dhcp-stats** skončí s návratovou hodnotou 0 v případě úspěchu a s návratovou hodnotou >0 v případě chyby.

4.3 Příklad spuštění

Zpracování DHCP komunikace z *pcap* souboru `homenetwork.pcap` pro síťové prefixy 192.168.1.0/24, 192.168.0.0/22 a 172.16.32.0/24:

```
$ ./dhcp-stats -r homenetwork.pcap 192.168.1.0/24 192.168.0.0/22 172.16.32.0/24
```

Monitorování DHCP komunikace na rozhraní `eth0` pro síťové prefixy 192.168.1.0/24, 192.168.0.0/22 a 172.16.32.0/24:

```
$ ./dhcp-stats -i eth0 192.168.1.0/24 192.168.0.0/22 172.16.32.0/24
```

4.4 Výstup

```
$ ./dhcp-stats -i eth0 192.168.1.0/24 192.168.0.0/22 172.16.32.0/24
IP-Prefix Max-hosts Allocated addresses Utilization
192.168.0.0/22 1022 123 12.04%
192.168.1.0/24 254 123 48.43%
172.16.32.0/24 254 15 5.9%
```

4.5 Logování

V případě, že počet alokovaných adres v prefixu překročí 50%, 75%, 90% a 95%, program tuto informaci zaloguje skrz standardní `syslog` [4] mechanismus do logu následovně:

```
prefix x.x.x.x/y exceeded 50% of allocations
prefix x.x.x.x/y exceeded 75% of allocations
prefix x.x.x.x/y exceeded 90% of allocations
prefix x.x.x.x/y exceeded 95% of allocations
```

5 Seznam odevzdaných souborů

- `dhcp-stats.1` – soubor ve formátu a syntaxi manuálové stránky [14].
- `dhcp-stats.c` – zdrojový soubor v programovacím jazyce C.
- `dhcp-stats.h` – hlavičkový soubor v programovacím jazyce C.
- `Makefile` – funkční soubor pro překlad zdrojového souboru.
- `manual.pdf` – soubor obsahující dokumentaci projektu ve formátu PDF.
- `README` – soubor ve formátu a syntaxi jazyka `Markdown` [1].

6 Literatura

- [1] CONE, M.: Getting Started. Markdown Guide, 2023, [citováno 2023-11-17]. Dostupné z: <https://www.markdownguide.org/getting-started/>
- [2] DROMS, R.: Dynamic Host Configuration Protocol. RFC 2131, 1997, [citováno 2023-11-15]. Dostupné z: <https://www.rfc-editor.org/info/rfc2131>
- [3] Free Software Foundation, Inc.: udp.h. The GNU C Library, 1982–2009, [citováno 2023-11-12]. Dostupné z: <https://sites.uclouvain.be/SystInfo/usr/include/netinet/udp.h.html>
- [4] Free Software Foundation, Inc.: Syslog Example. The GNU C Library Reference Manual, 1993–2023, [citováno 2023-11-16]. Dostupné z: https://www.gnu.org/software/libc/manual/html_node/Syslog-Example.html
- [5] Free Software Foundation, Inc.: GCC, the GNU Compiler Collection. GNU Project, 2023, [citováno 2023-11-10]. Dostupné z: <https://gcc.gnu.org>
- [6] KERRISK, M.: getopt(1). Linux manual page, 2023, [citováno 2023-11-11]. Dostupné z: <https://man7.org/linux/man-pages/man1/getopt.1.html>
- [7] LEON, J. D.: DevDungeon. Using libpcap in C, 2015, [citováno 2023-11-13]. Dostupné z: <https://www.devdungeon.com/content/using-libpcap-c>
- [8] PADALA, P.: Initialization. NCURSES Programming HOWTO, 2005, [citováno 2023-11-17]. Dostupné z: <https://tldp.org/HOWTO/NCURSES-Programming-HOWTO/init.html>
- [9] SONG, J.: SYMBEXNET. Packet format for DHCP, 2014, [citováno 2023-11-17]. Dostupné z: https://www.researchgate.net/figure/Packet-format-for-DHCP_fig10_263813522
- [10] The Open Group: math.h. Mathematical declarations, 2004, [citováno 2023-11-12]. Dostupné z: <https://pubs.opengroup.org/onlinepubs/007904875/basedefs/math.h.html>
- [11] The Tcpdump Group: Tcpdump & Libpcap. Programming with pcap, 1999–2023, [citováno 2023-11-15]. Dostupné z: <https://www.tcpdump.org/pcap.html>
- [12] Wikipedia contributors: Classless Inter-Domain Routing. Wikipedia, The Free Encyclopedia, 2023, [citováno 2023-11-15]. Dostupné z: https://en.wikipedia.org/w/index.php?title=Classless_Inter-Domain_Routing&oldid=1184914410
- [13] Wikipedia contributors: Dynamic Host Configuration Protocol. Wikipedia, The Free Encyclopedia, 2023, [citováno 2023-11-14]. Dostupné z: https://en.wikipedia.org/w/index.php?title=Dynamic_Host_Configuration_Protocol&oldid=1178393279
- [14] WIRZENIUS, L.: Example manual page. Writing manual pages, 2019, [citováno 2023-11-17]. Dostupné z: <https://liw.fi/manpages/>